

DA6400 : Reinforcement Learning (January–May 2026)

Programming Assignment 3 Submission

Ananya Lakshmi Ravi* S Shriprasad*

NS26Z145 DA25E054

*Equal Contribution

▷ [Source Code](#)

1 Section 2.1: Pendulum

1.1 Environment and Reward Design

We modify `Pendulum-v1` to stabilize at a target angle θ_{target} . The observation is $s_t = (\cos \theta_t, \sin \theta_t, \dot{\theta}_t)$ and the action u_t is the applied torque. Episodes are fixed-length (1000 steps) with no terminal state. We designed the reward:

$$r_t = -\text{reward_scale} \left(w_\theta e_t^2 + w_\omega \dot{\theta}_t^2 + w_u u_t^2 \right), \quad e_t = \text{wrap}(\theta_t - \theta_{\text{target}}), \quad (1)$$

with $w_\theta = 1.0$, $w_\omega = 0.1$, $w_u = 0.001$. The reward penalizes three quantities:

- $w_\theta e_t^2$: squared wrapped angle error — makes θ_{target} the unique optimum.
- $w_\omega \dot{\theta}_t^2$: angular velocity — prevents swinging through the target; encourages stabilization.
- $w_u u_t^2$: torque — discourages aggressive control, promotes smooth actuation.

The reward is dense (non-zero at every step). The target angle enters only via e_t , so the reward structure is identical across all targets.

We implemented SAC with tanh-squashed Gaussian policy, clipped double Q-learning, target networks, reparameterized actor updates, Adam optimizer, and automatic entropy temperature tuning (unless specified otherwise). All curves plot mean return vs. environment timesteps with 95% CI over 15 seeds, using 20 offline evaluation episodes every 10K steps.

1.2 SAC Across Target Angles

Figure 1 shows auto- α SAC for $\theta_{\text{target}} \in \{0, -10, 30, -60, 90, -90, 120, -150\}^\circ$.

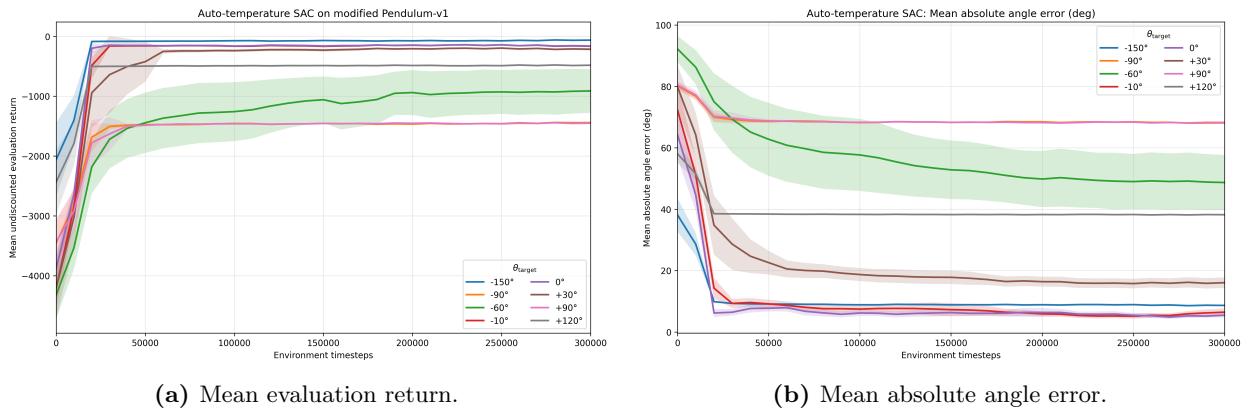


Figure 1: Auto- α SAC across target angles (mean $\pm$ 95% CI).

Easy targets (0° , -10° , -150°) converge to low angle error ($< 9^\circ$) and high target occupancy (> 0.95). These targets require little or no sustained torque against gravity.

Hard targets ($\pm 90^\circ$, -60° , 120°) do not converge to the exact target angle. The reason is the default torque bound. In Gymnasium’s Pendulum-v1 ($m = 1$, $l = 1$, $g = 10$), the approximate static holding torque for a target angle is

$$|u_{\text{hold}}| = \frac{mgl}{2} |\sin \theta_{\text{target}}| = 5 |\sin \theta_{\text{target}}|.$$

The default action bound is $|u| \leq 2$. For targets such as $\pm 90^\circ$ ($|u_{\text{hold}}| = 5.0$), -60° and 120° ($|u_{\text{hold}}| = 4.33$), the required holding torque exceeds the available range. The policy still learns useful behavior—it damps oscillations and reduces angular velocity—but it stabilizes at a compromise angle closer to the natural equilibrium rather than exactly at the target.

The -60° target shows high seed-to-seed variability (-903.54 ± 368.75): some seeds discover a viable control strategy while others settle into a low-effort equilibrium. Its mean curve should be interpreted together with the wide confidence interval.

Target occupancy is the fraction of evaluation timesteps for which the wrapped angle error lies within a 10° tolerance.

Table 1: Final auto- α SAC results across target angles.

θ_{target}	Final Return	Angle Error (deg)	Target Occupancy	Mean Sq. Torque
-150°	-72.79 ± 3.04	8.94	0.971	3.563
-90°	-1449.92 ± 6.81	68.23	0.0007	3.512
-60°	-903.54 ± 368.75	48.39	0.002	2.979
-10°	-156.64 ± 7.31	6.37	0.958	0.919
0°	-153.08 ± 7.00	5.13	0.965	0.209
30°	-206.43 ± 20.26	15.76	0.130	2.131
90°	-1468.75 ± 9.92	68.56	0.0004	3.464
120°	-496.51 ± 2.00	38.39	0.003	3.618

1.3 Optimal Behavior

The optimal behavior has **two aspects**:

1. **Reaching:** from the random initial state, move the pendulum toward θ_{target} . Depending on the target, this may require one or more swings to build momentum.
2. **Stabilization:** once near the target, maintain low wrapped angle error and low angular velocity for the remainder of the 1000-step episode. A policy that merely passes through the target without staying is suboptimal.

Return alone does not distinguish these aspects. A policy can achieve low angular velocity yet stabilize at the wrong angle. We therefore also examine angle error, target occupancy, angular velocity, and torque (Appendix A). For easy targets, the policy achieves both reaching and stabilization. For hard targets, the policy often damps oscillations effectively but stabilizes at a compromise angle, because exact target holding requires more torque than the default action bound allows.

We ran a diagnostic with wider torque $[-6, 6]$ for -60° , which achieved near-perfect tracking (2–3° error, occupancy 0.98). This strongly supports the interpretation that the default -60° difficulty is mainly due to torque-limit and reward-tradeoff constraints, rather than SAC being unable to learn useful control (Appendix B).

Final policy snapshots for Pendulum target angles

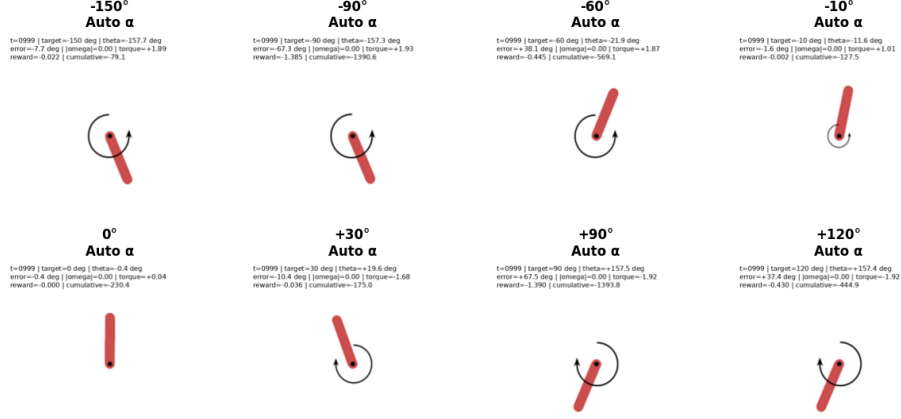


Figure 2: Final policy snapshots at $t = 999$ for all target angles (auto- α). Easy targets (0° , -10° , -150°) show near-zero angle error. Hard targets ($\pm 90^\circ$, 120°) stabilize at a compromise angle due to torque constraints. Note that all policies achieve $|\dot{\theta}| = 0$ (successful damping), but only feasible targets achieve low angle error.

1.4 Manual vs. Automatic α

We searched $\alpha \in \{0.01, 0.03, 0.05, 0.1, 0.2, 0.5\}$ for $\theta \in \{-60, 90, 120, -150\}^\circ$. Selected: $\alpha_{\text{mnl}} = 0.03$ for -60° ; $\alpha_{\text{mnl}} = 0.01$ for 90° , 120° , and -150° . This selection required $4 \times 6 \times 3 = 72$ pilot runs (4 targets, 6 alpha values, 3 tuning seeds) before the final comparison, making manual tuning significantly more expensive than automatic tuning.

Manual α vs automated α on selected Pendulum targets

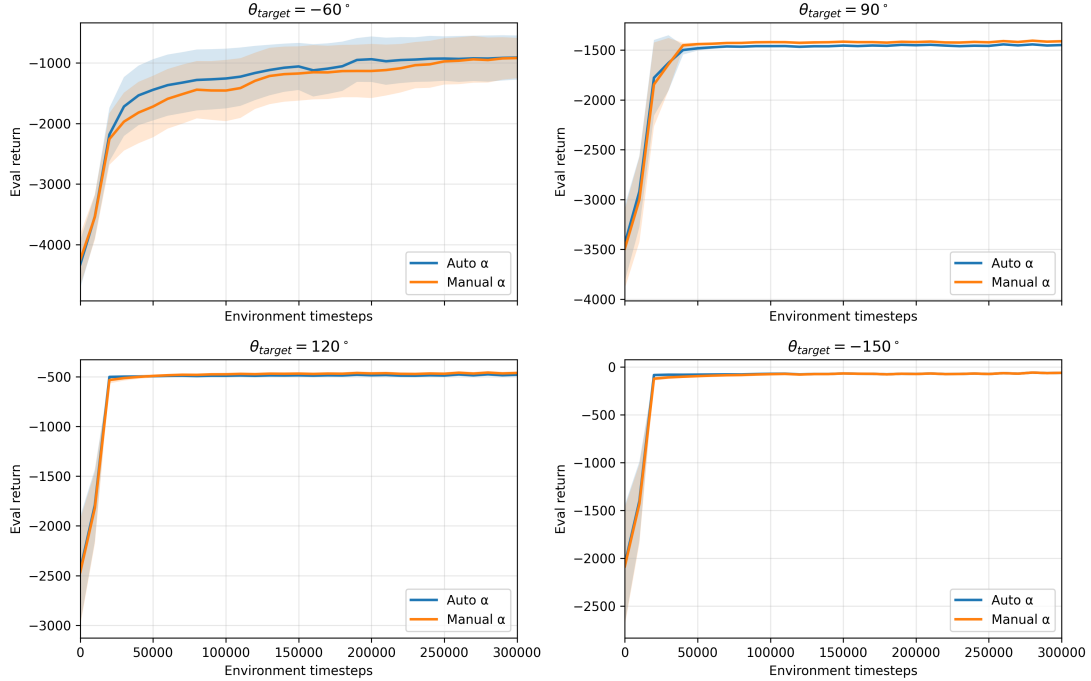


Figure 3: Manual α vs. auto-tuning (mean $\pm$ 95% CI).

Manual α gives modest gains for 90° (-1429 vs -1469) and 120° (-478 vs -497). For -150° , both methods are essentially equivalent. For -60° , both show wide confidence intervals, making the

comparison inconclusive. Auto-tuning is a strong default: it avoids the 72-run search cost and remains competitive across all targets. Manual tuning can help specific targets when one is willing to invest in the additional pilot experiments.

1.5 Reward Scaling

For $\theta = 90^\circ$ with scales $0.1\times$ and $10\times$:

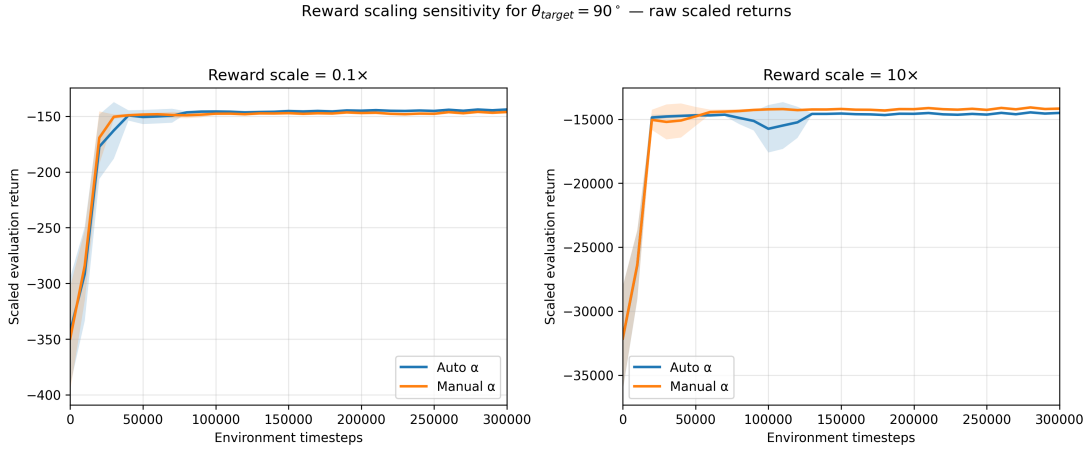


Figure 4: Reward scaling: auto vs. manual α at $0.1\times$ and $10\times$.

Does manual α adapt? No. Manual α was tuned at reward scale $1\times$. Scaling the reward by $10\times$ makes the entropy term $\alpha\mathcal{H}$ relatively ten times weaker, reducing exploration. Scaling by $0.1\times$ makes it ten times stronger, potentially over-exploring. In both cases the effective reward-entropy balance shifts away from the tuned optimum.

Does auto α adapt? Mostly yes. Automatic tuning adjusts the entropy temperature to maintain target entropy regardless of reward scale. However, reward scaling also changes critic target magnitudes and gradient scales, which affect learning dynamics independently of entropy. At $0.1\times$: auto- α achieves -145.3 , manual achieves -147.7 . At $10\times$: manual achieves -14359.8 , auto achieves -14674.9 —the auto curve shows a temporary mid-training dip from large critic gradients. Automatic tuning improves robustness but does not fully remove sensitivity to reward magnitude.

2 Section 2.2: Lunar Lander

2.1 Experimental Setup

We used `LunarLander-v3` in both continuous and discrete action settings. For the continuous-action experiments, SAC uses a tanh-squashed Gaussian actor, twin Q-critics, clipped double-Q targets, target networks, reparameterized actor updates, Adam optimizer, and automatic entropy-temperature tuning. The main hyperparameters were hidden layers (256, 256), learning rates 3×10^{-4} , discount factor $\gamma = 0.99$, target update coefficient $\tau = 0.005$, batch size 256, replay buffer size 10^6 , and 10K initial random exploration steps. Evaluation was done every 10K environment steps using 20 deterministic offline episodes.

Table 2: Summary of LunarLander experiments. Values are final evaluation returns.

Experiment	Action space	Steps	Final return	Main conclusion
Continuous SAC, auto- α	Continuous	300K	176.6 ± 88.1	Learns, but seed-sensitive
Hover SAC, fixed $\alpha = 0.01$	Continuous	600K	167.3 ± 84.5	Weak recovery after switch
Hover SAC, auto- α	Continuous	600K	271.1 ± 5.4	Strongest and most stable
Discrete SAC	Discrete	250K	-758.2 ± 0.0	Unstable / failed run
DQN	Discrete	250K	128.2 ± 86.8	Learns steadily

2.2 Continuous SAC with Automated Temperature Tuning

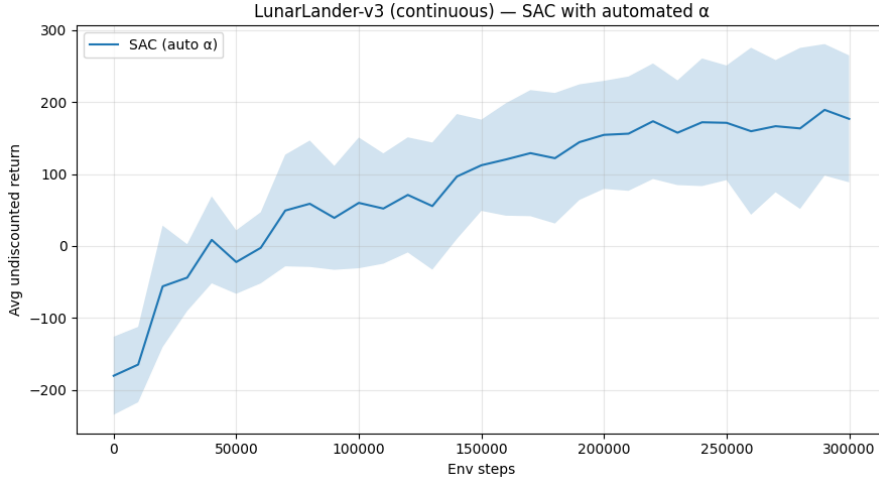
**Figure 5:** Continuous SAC on LunarLander-v3 with automated temperature tuning.

Figure 5 shows that continuous SAC learns meaningful control behavior. The average return improves from strongly negative values to approximately 176.6 by 300K steps. The behavior can be divided into three stages. In the initial stage, the policy fires engines randomly, rotates excessively, and crashes. In the intermediate stage, the agent learns to stabilize and slow its descent—a common local optimum where avoiding crashes gives better return than landing poorly. In the final stage, successful seeds learn controlled descent and attempt to land between the flags. The wide confidence interval shows this final behavior is not equally strong for all seeds.

2.3 Hover-Box Reward Switch

We modified the continuous LunarLander environment by adding a one-time hover reward. Initially +200 for entering the box $|x| < 0.1$, $0.4 < |y| < 0.6$, then changed to -100 . We compared fixed $\alpha = 0.01$ and automated α .

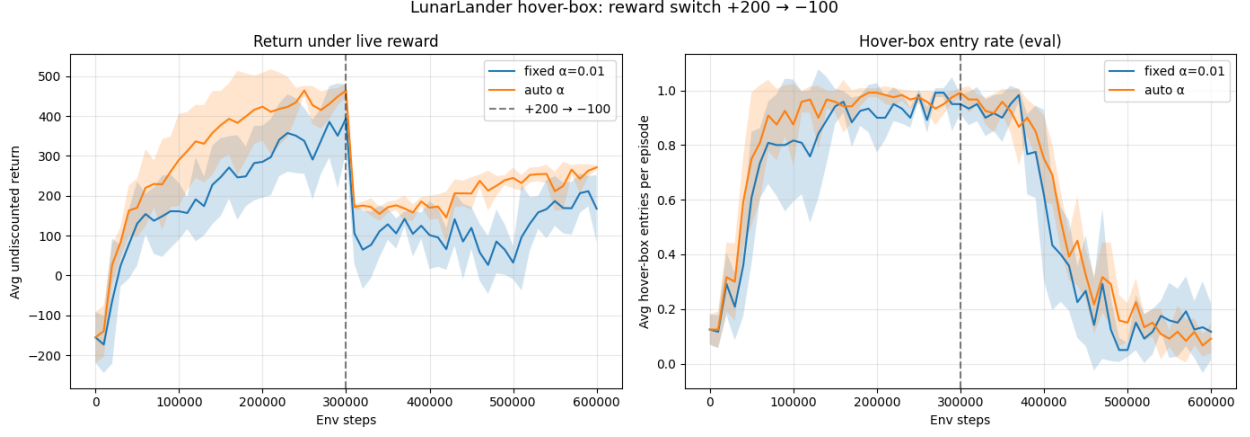


Figure 6: Hover-box reward-switch experiment. Left: evaluation return. Right: hover-box entry rate. Dashed line marks the switch from +200 to -100.

Before the switch, both agents exploit the hover box. After the switch, auto- α recovers to 271.1 ± 5.4 , while fixed- α remains noisy at 167.3 ± 84.5 . This is the strongest result in this section.

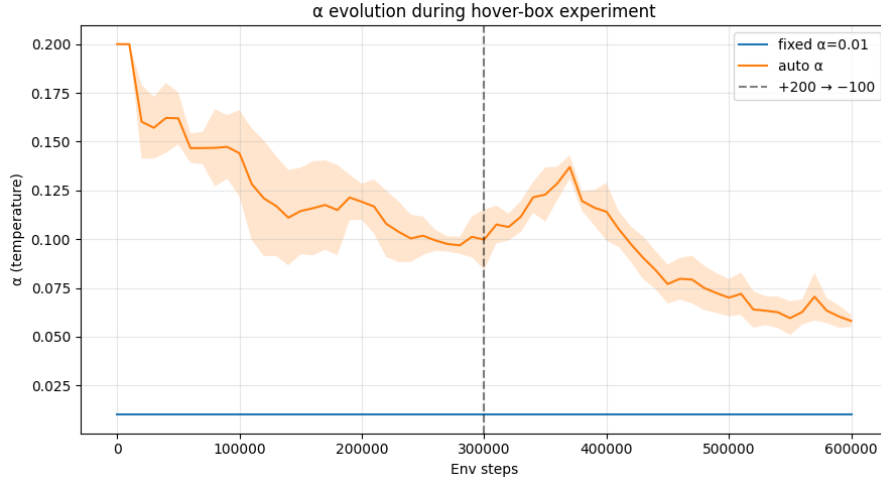


Figure 7: Evolution of α during the hover-box experiment. Auto- α adapts around the reward switch.

Figure 7 explains why: auto- α increases entropy around the switch, enabling exploration away from the old local optimum. This supports the maximum-entropy interpretation of SAC:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_t r_t + \alpha \mathcal{H}(\pi(\cdot|s_t)) \right].$$

Entropy regularization discourages premature determinism and improves robustness when the reward landscape changes.

2.4 Discrete SAC Formulation

For the discrete-action version, the Gaussian policy is replaced by a categorical policy $\pi(a|s)$. Since the action set is small, expectations are computed exactly:

$$y = r + \gamma(1-d) \sum_{a'} \pi(a'|s') \left[\min_i Q_i^{\text{targ}}(s', a') - \alpha \log \pi(a'|s') \right].$$

$$J_{\pi} = \mathbb{E}_s \left[\sum_a \pi(a|s) \left(\alpha \log \pi(a|s) - \min_i Q_i(s, a) \right) \right].$$

2.5 Discrete SAC vs. DQN

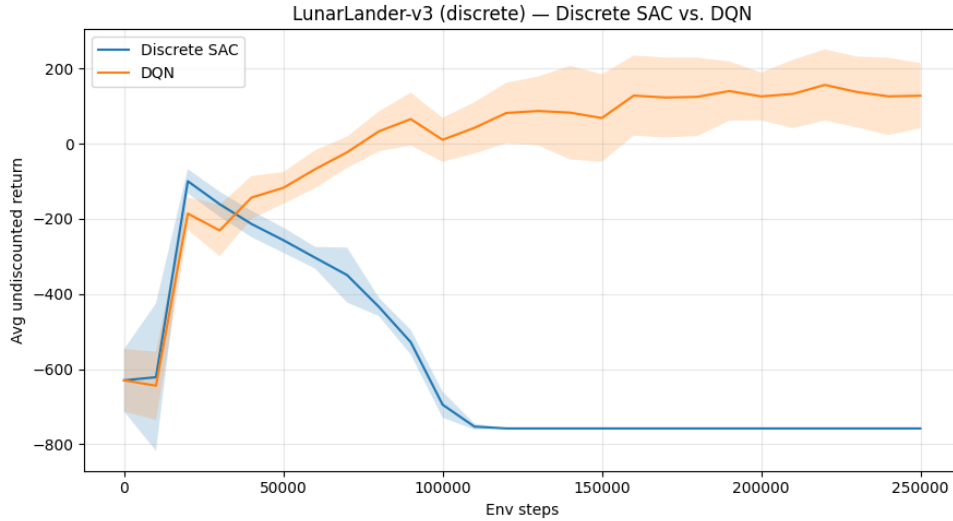


Figure 8: Discrete SAC vs. DQN on discrete-action LunarLander-v3.

DQN learns steadily and reaches approximately 128.2 ± 86.8 . Our Discrete SAC run collapses to -758.2 after around 100K steps due to alpha explosion. We investigated the root cause: with `target_entropy_ratio=0.50`, the target entropy is $0.50 \times \ln(4) = 0.693$, but actual policy entropy stabilizes near 1.35 (close to maximum entropy $\ln(4) = 1.386$). Since entropy consistently exceeds the target, the alpha update pushes α upward in a positive feedback loop ($\alpha : 0.2 \rightarrow 8.5M$), causing critic loss to diverge.

We attempted a fix by setting `target_entropy_ratio=0.98` (target entropy = 1.36, matching the observed entropy). The agent initially showed improvement (return rose from -580 to -17 at 10K steps), but subsequently collapsed to the same -758.2 level. This suggests the instability is not purely due to the entropy target mismatch—the critic loss scaling and actor-critic coupling under large Q-values also contribute to the divergence. A more robust fix would likely require gradient clipping, alpha clamping, or a different critic architecture.

Preference for discrete actions. Based on our results, DQN is clearly preferred for discrete LunarLander:

- **Performance:** DQN achieves $+128$ mean return vs. Discrete SAC’s -758 .
- **Sample efficiency:** DQN improves steadily throughout training; Discrete SAC collapses after initial progress.
- **Ease of implementation:** DQN requires only a Q-network, target network, and ϵ -greedy exploration. Discrete SAC adds a categorical actor, twin critics, entropy temperature, and entropy-target tuning—all of which increase sensitivity to hyperparameters.

Discrete SAC remains theoretically attractive for its principled exploration via entropy maximization, but in practice it requires careful tuning of the entropy target and alpha dynamics that DQN avoids entirely.

3 Section 2.3: Reacher

3.1 Setup

We used `reacher-easy` from the DeepMind Control Suite. The task consists of a two-joint robotic arm, and the desired behavior is to reach the target sphere quickly and remain there. We trained

three SAC agents corresponding to three reward formulations:

$$R_a(s, a) = \begin{cases} 1, & \text{fingertip in target,} \\ -\|x_{\text{goal}} - x_{\text{finger}}\| - \|a\|^2, & \text{otherwise,} \end{cases} \quad (2)$$

$$R_b(s, a) = \begin{cases} 1, & \text{fingertip in target,} \\ 0, & \text{otherwise,} \end{cases} \quad (3)$$

$$R_c(s, a) = -1 \quad \text{per step until the target is reached with low velocity.} \quad (4)$$

For R_a and R_b , episodes are fixed-length with horizon 1000. For R_c , if the agent does not reach the target within 1000 steps, we reset the arm while keeping the same target and apply a timeout penalty of -20 . All agents trained for 500K steps using the same SAC setup. For cross-evaluation, each trained policy was evaluated under all three reward formulations.

3.2 Self-Evaluation

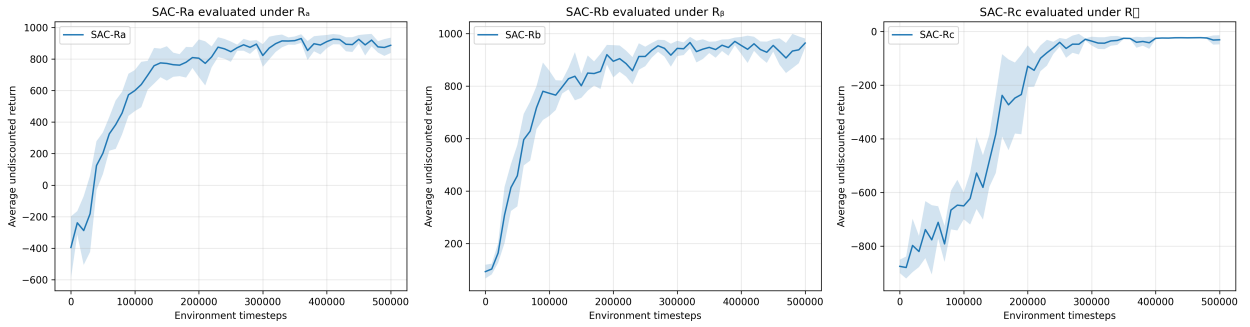


Figure 9: Self-evaluation: each agent evaluated under the reward it was trained with.

SAC- R_b learns fastest and reaches the highest self-evaluation return (~ 950). SAC- R_a converges steadily to ~ 900 . SAC- R_c starts very negative but improves toward zero (better for R_c since the reward is -1 per step until success). Learning is faster for R_a and R_b because they provide informative signal before the target is reached, while R_c gives a harsh constant -1 until the agent discovers successful reaching.

3.3 Final-Policy Behavioral Metrics

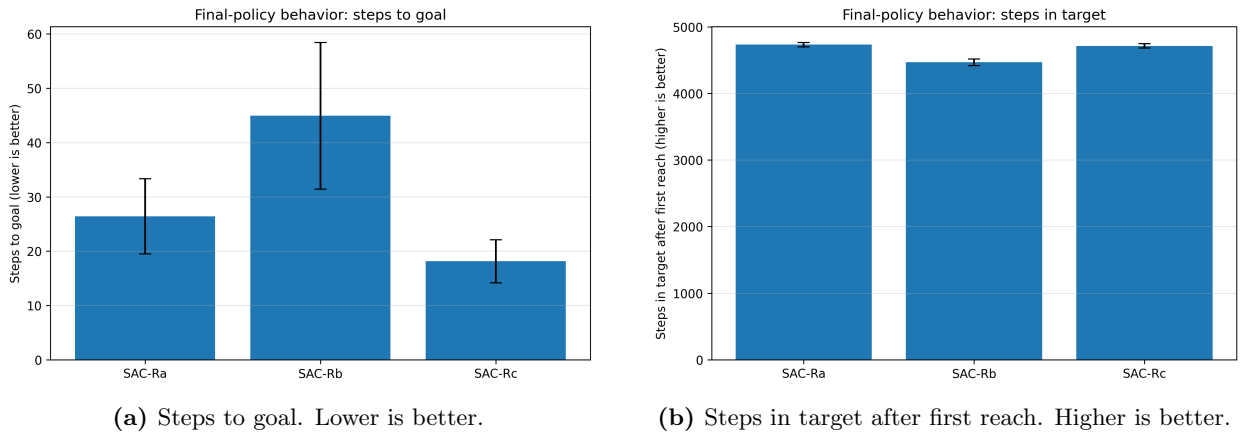


Figure 10: Final-policy diagnostics using 500 evaluation episodes of length 5000.

SAC- R_c reaches the target fastest (~ 18 steps), as expected because R_c directly penalizes time. SAC- R_a is second (~ 25 – 30 steps), while SAC- R_b is slower (~ 45 steps) with higher variance. For

steps-in-target, all three policies remain inside the target for most of the 5000-step horizon. SAC- R_c is best for fast reaching, while SAC- R_a gives the most balanced reach-and-stay behavior.

3.4 Cross-Evaluation

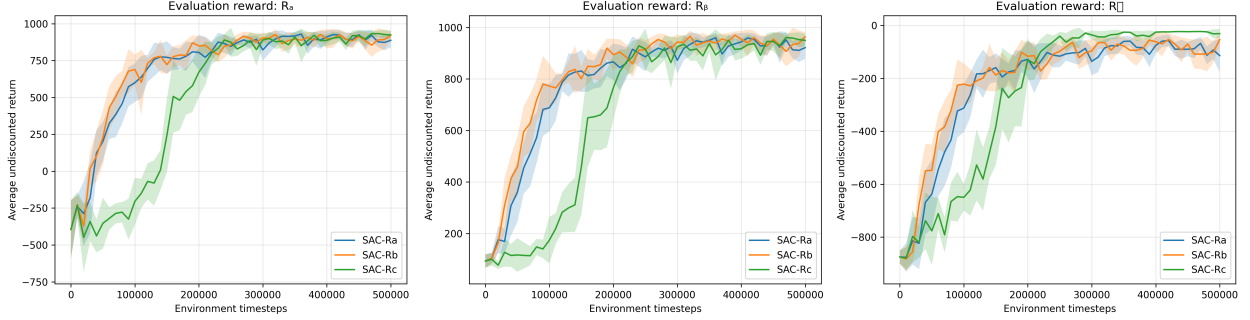


Figure 11: Cross-evaluation: each subplot fixes the evaluation reward and compares SAC- R_a , SAC- R_b , and SAC- R_c .

Under R_a and R_b evaluation, all three policies converge to similar final returns—the rewards are partially aligned since all encode target-reaching behavior. Under R_c evaluation, SAC- R_c achieves the best final performance because it directly optimizes time-to-goal. SAC- R_a and SAC- R_b improve substantially but remain slightly worse under the time-to-goal metric.

Overall comparison. In terms of ease of specification, R_b is simplest (binary target detection). In terms of learning efficiency, R_a and R_b learn faster early on. In terms of achieving the desired behavior, R_c is best for reaching quickly, while R_a gives the best balanced behavior (reaching, staying, smooth control). We recommend R_a in practice for stable performance, and R_c when minimizing time-to-goal is the primary objective.

4 Section 3 (Bonus): PEBBLE

PEBBLE learns a reward model from pairwise preferences via the Bradley-Terry model:

$$P_\psi(\sigma^0 \succ \sigma^1) = \frac{\exp(\sum_t r_\psi(s_t^0, a_t^0))}{\exp(\sum_t r_\psi(s_t^0, a_t^0)) + \exp(\sum_t r_\psi(s_t^1, a_t^1))}.$$

A reward ensemble of 3 MLPs is trained with cross-entropy loss on teacher labels; SAC then optimizes the learned reward. The teacher is simulated using ground-truth cumulative return. After each reward-model update, all replay buffer rewards are relabelled with the ensemble prediction.

4.1 PEBBLE on Pendulum

We compare PEBBLE (budget=200) against SAC with ground-truth reward for $\theta \in \{0, -60, 90, 120, -150\}^\circ$.

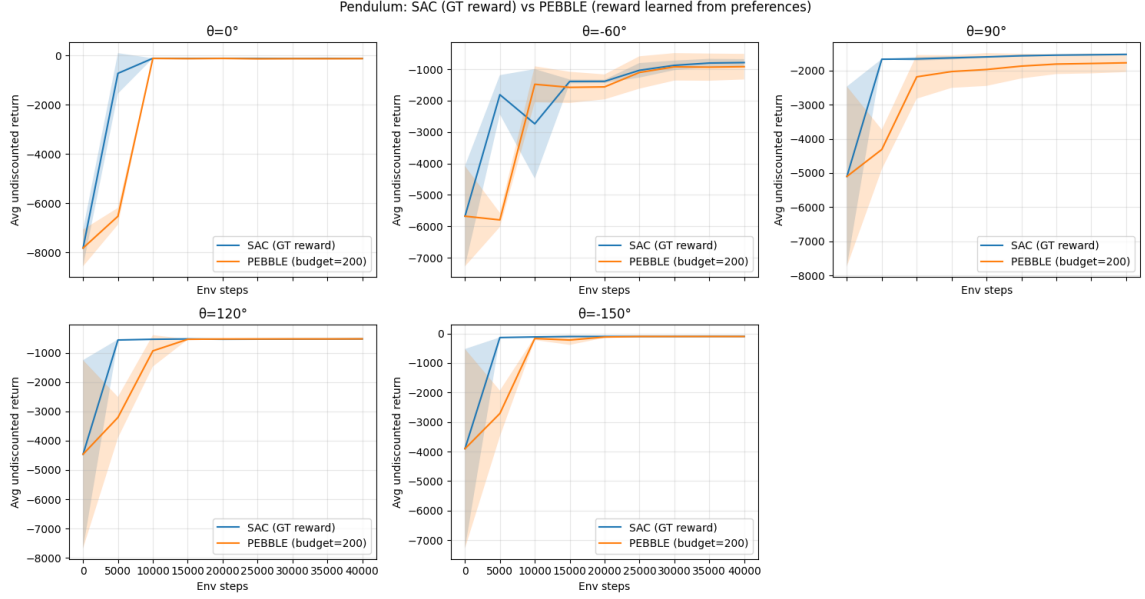


Figure 12: PEBBLE vs SAC (GT reward) on Pendulum for five target angles. Y-axis is undiscounted ground-truth return.

PEBBLE asymptotically matches SAC-GT on all five target angles. For easy targets (0° , -150°), PEBBLE converges within $\sim 10\text{K}$ steps to the same level as SAC-GT. For harder targets (-60° , 90° , 120°), both methods converge to similar final returns, though PEBBLE starts slower because (a) the first $\sim 5\text{K}$ steps use intrinsic-only exploration, and (b) the learned reward model is initially less informative than the hand-crafted quadratic. The similar final performance across both methods indicates that 200 preference queries are sufficient for the Pendulum reward structure.

4.2 Feedback Budget Ablation

We ablate the feedback budget for $\theta = 90^\circ$ with budgets $B \in \{50, 200\}$ and include SAC-GT as reference.

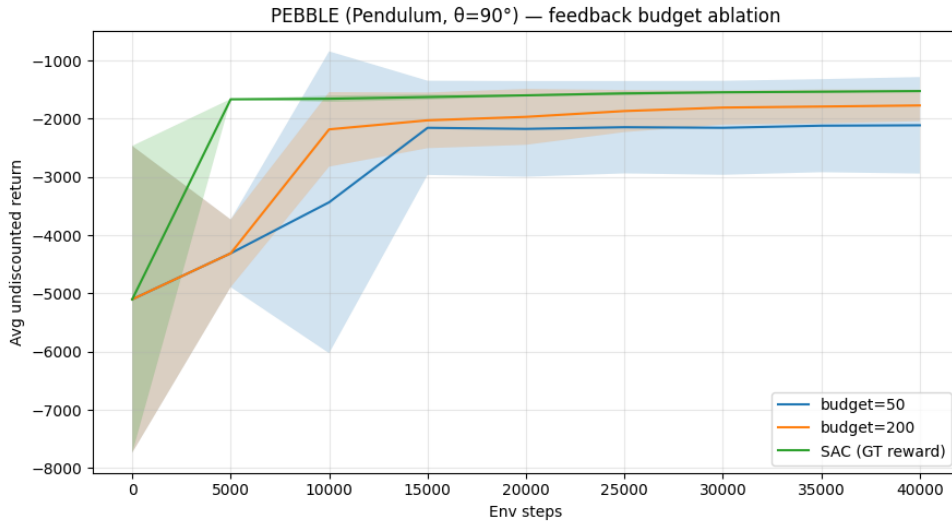


Figure 13: Budget ablation ($\theta = 90^\circ$). SAC-GT converges fastest; $B = 200$ nearly matches it; $B = 50$ is noisier with wider CI.

With $B = 200$, PEBBLE nearly matches SAC-GT by the end of training, confirming that moderate feedback is sufficient to learn the Pendulum reward structure. With $B = 50$, the reward model

receives too few labels, resulting in a noisier approximation and wider confidence intervals. The policy still improves but converges to a lower final return. This shows that preference quality and quantity directly affect reward-model accuracy and downstream policy performance, with diminishing returns beyond ~ 200 queries for this task.

4.3 PEBBLE on Reacher

We use three simulated teachers based on R_a , R_b , and R_c (budget=200).

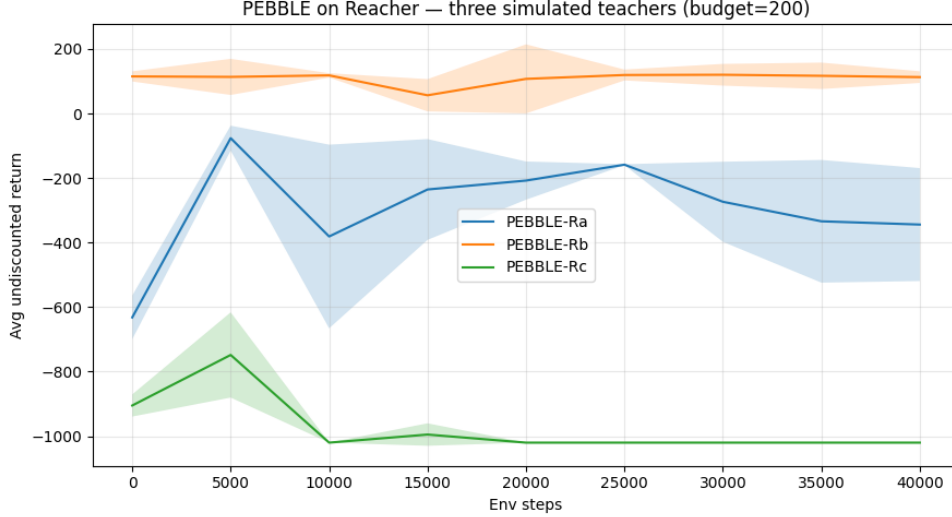


Figure 14: PEBBLE on Reacher with three simulated teachers. Y-axis is ground-truth return under each teacher’s reward.

PEBBLE- R_b achieves the best and most stable performance (~ 120 return), learning quickly and maintaining tight confidence intervals. R_b is a binary indicator (1 if in target, 0 otherwise). Once the agent discovers any successful segment, the preference labels become highly informative: successful segments are clearly preferred over unsuccessful ones. This makes reward learning straightforward.

PEBBLE- R_a is unstable and remains negative (~ -200). Although R_a is dense (distance-to-target), the preferences mix distance improvement with action-cost penalties, making the labels more complex. The reward model struggles to disentangle these competing objectives, leading to unstable policy optimization.

PEBBLE- R_c collapses to approximately -1020 (the timeout return). Since $R_c = -1$ per step until termination, most unsuccessful segments receive nearly identical cumulative returns ($-T$). This makes preference labels weakly informative—the teacher cannot distinguish between two segments that both fail, even if one is closer to succeeding. The reward model therefore learns a near-constant function, and the policy makes no progress.

Takeaway: Simpler, more discriminative reward formulations make better preference teachers. R_b ’s binary structure produces the clearest preference signal, while R_c ’s time-penalty structure produces the least informative comparisons.

A Pendulum Behavior Diagnostics

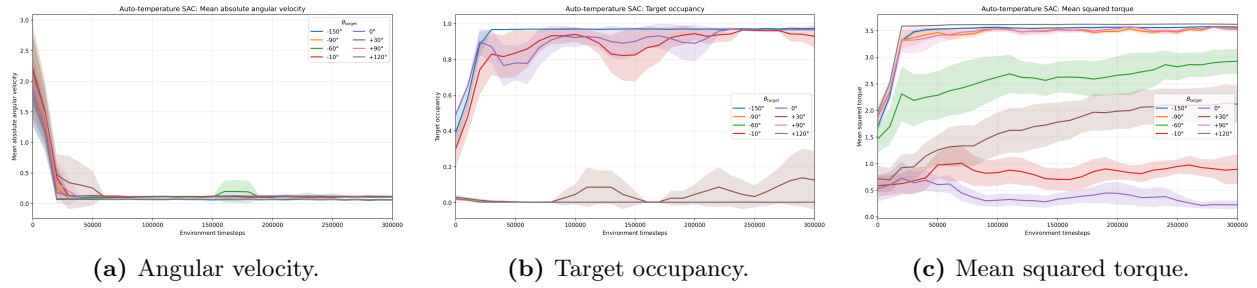


Figure 15: Behavior diagnostics for auto- α SAC across targets.

B Torque-Limit Diagnostic (-60°)

Table 3: Effect of torque range on $\theta_{\text{target}} = -60^\circ$.

Setting	Final Return	Angle Error	Occupancy	Mean Sq. Torque
Default $[-2, 2]$	-903.54 ± 368.75	48.39°	0.002	2.979
Wider $[-6, 6]$	≈ -49	$\approx 2-3^\circ$	≈ 0.98	≈ 18.7

Holding at -60° requires $|u_{\text{hold}}| = 5|\sin(-60^\circ)| \approx 4.33$, which exceeds the default action limit $|u| \leq 2$. Exact static holding at -60° is therefore not feasible under the default torque range. With the wider range $[-6, 6]$, the required holding torque becomes feasible and the policy achieves near-perfect tracking. This supports the torque-authority interpretation.

Table 4: Static holding torque feasibility under default bound $|u| \leq 2$.

θ_{target}	$ u_{\text{hold}} = 5 \sin \theta $	Feasible?
0°	0.00	Yes
-10°	0.87	Yes
30°	2.50	Borderline
-60°	4.33	No
90°	5.00	No
-90°	5.00	No
120°	4.33	No
-150°	2.50	Borderline

Borderline cases (30° , -150°) can still achieve good occupancy because the 10° tolerance allows nearby angles requiring slightly lower holding torque, and dynamic regulation can keep the pendulum close to the target region.

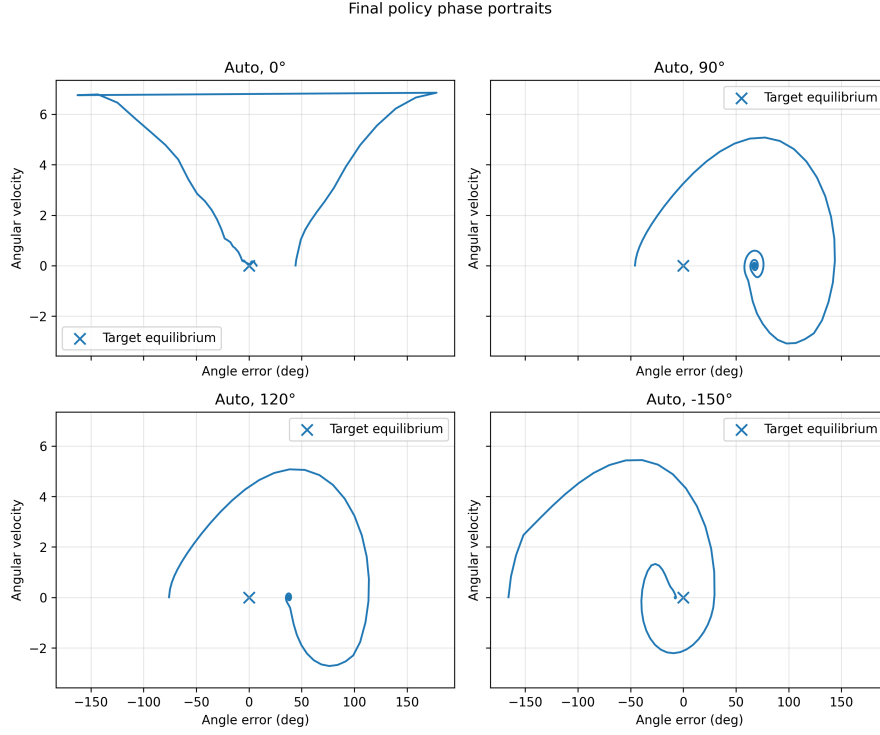


Figure 16: Phase portraits for selected final policies.

C Additional Reacher Diagnostics

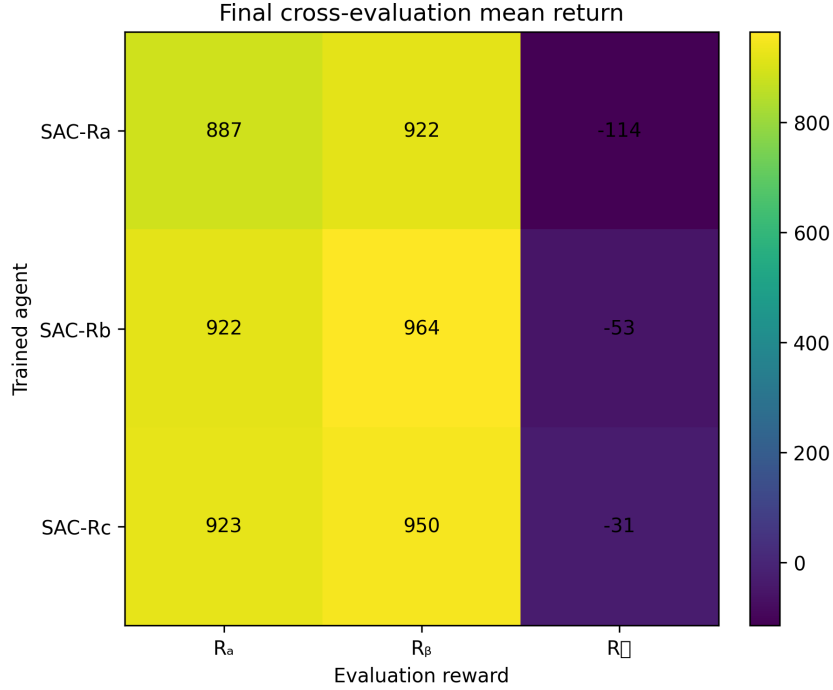


Figure 17: Final cross-evaluation mean returns for Reacher. Rows indicate the training reward and columns indicate the evaluation reward.

The heatmap summarizes the final cross-evaluation point. All policies obtain high final returns under R_a and R_b , showing that all three reward formulations learn target-reaching behavior. Under R_c , SAC- R_c achieves the best value, confirming that the time-to-goal reward is most aligned with

fast reaching.

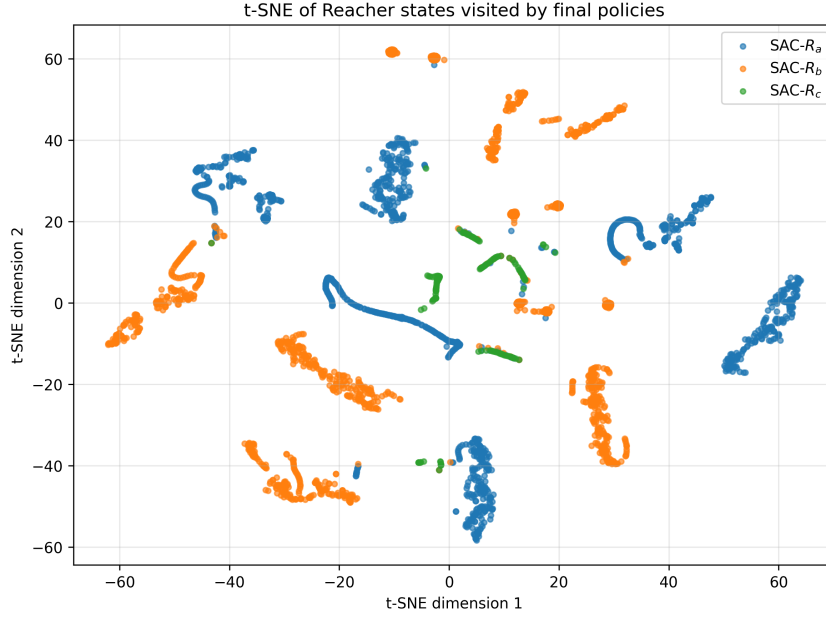


Figure 18: t-SNE projection of states visited by final Reacher policies. This visualization is qualitative only.

The t-SNE visualization shows that policies trained under different reward formulations visit different projected regions of the state space. This suggests that reward design affects trajectory distribution. Since t-SNE is only a nonlinear projection, we use it only as supporting qualitative evidence; the main conclusions are based on learning curves and final-policy behavioral metrics.

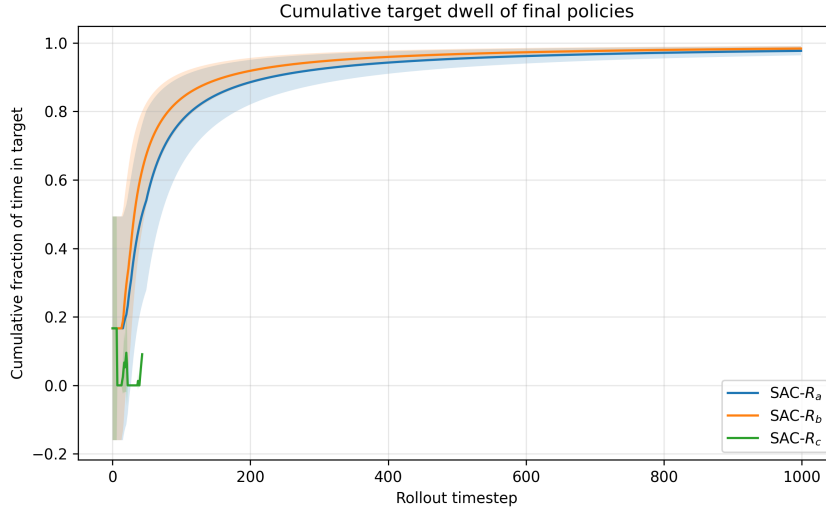


Figure 19: Cumulative fraction of rollout time spent inside the target. SAC- R_c terminates early under its evaluation rule after reaching the target with low velocity, so its shorter curve indicates early success rather than poor dwell behavior.

The dwell plot shows that SAC- R_a and SAC- R_b quickly reach the target and spend most of the rollout inside it. SAC- R_c is not directly comparable in this plot because the R_c evaluation episode terminates once the target is reached with sufficiently low velocity. Therefore, the short SAC- R_c curve should be interpreted as early successful termination.